

CS6.201 — Introduction to Software Systems

End-Semester Exam — Complete Answer Key & Marking Scheme

Total Marks	120	Duration	3 hours
-------------	-----	----------	---------

Q	1	2	3	4	5	6	7	8	9	10	11	12	Total
Marks	8	8	3	6	3	15	18	12	8	12	15	12	120

Question 1 — isAnagram + Time Complexity

/8 marks

Part (a) — isAnagram implementation [5 marks]

The function must: lowercase both strings, strip spaces and punctuation, then compare the cleaned strings to determine if they are anagrams.

Reference Solution:

```
import re

def isAnagram(str1, str2):
    clean1 = re.sub(r'[^\a-z]', '', str1.lower()) # lowercase + remove non-alpha
    clean2 = re.sub(r'[^\a-z]', '', str2.lower())
    return sorted(clean1) == sorted(clean2) # O(n log n) comparison
```

Alternative accepted: Using `Counter(clean1) == Counter(clean2)` gives $O(n)$ and is equally valid.

Criterion	Marks
Correct function signature and return value (True/False)	1
Correctly converts to lowercase	0.5
Correctly removes spaces	0.5
Correctly removes punctuation	0.5
Correctly compares cleaned texts (sorting or frequency/Counter approach)	2
Deduction: minor logic/syntax errors	−0.5 to −1

Part (b) — Time Complexity [3 marks]

Criterion	Marks
States the correct time complexity with brief justification	2 + 1
→ $O(n)$ using Counter/frequency array — full 2 marks	2
→ $O(n \log n)$ using sorting — deduct 0.5 (accepted but not optimal)	1.5
→ $O(n^2)$ or worse — deduct 1	1
Brief explanation of why (e.g. 'sorting is $O(n \log n)$ ', 'Counter is $O(n)$ ')	1

Question 2 — Kaprekar's Routine

/8 marks

Clarifications

- The example says 8991 reaches 6174 in *two* steps, but the computation shown is three subtractions. Accept **3 steps** as the correct answer for 8991 — do not penalise either interpretation. Similarly for 6264: accept 2 or 3 steps

depending on interpretation.

- Zero-padding (`zfill(4)`) is not required but is also fully accepted.
- Returning the result instead of printing it is fully accepted.

Reference Solution A — Using a seen set (general):

```
def kaprekar_steps(n):
    seen = set()
    steps = 0
    while n not in seen:
        seen.add(n)
        s = str(n) # or str(n).zfill(4)
        desc = int(''.join(sorted(s, reverse=True)))
        asc = int(''.join(sorted(s)))
        n = desc - asc
        steps += 1
    return steps - 1 # or print(steps - 1)
```

Reference Solution B — Stop at 6174 (also accepted):

```
def kaprekar_steps(n):
    steps = 0
    while n != 6174:
        s = str(n) # or str(n).zfill(4)
        desc = int(''.join(sorted(s, reverse=True)))
        asc = int(''.join(sorted(s)))
        n = desc - asc
        steps += 1
    return steps # or print(steps)
```

Criterion	Marks
Correctly converts the number to a digit sequence/string for sorting	1
Correctly sorts digits in descending order	1
Correctly sorts digits in ascending order	1
Converts sorted digit strings back to integers and computes the difference	1.5
Correctly repeats the process in a loop	1
Correctly counts the number of iterations/steps	1
Stops at fixed point 6174 OR detects repetition via a seen set/list	1
Produces required output (print or return); works on given examples	0.5

Question 3 — NumPy Boolean Indexing

/3 marks

Model Answer:

```
data = [10 20 30 40 50] and mask = [False False True True True]
```

Explanation: `mask = data > 25` creates a boolean array where each element is compared against 25, yielding `[False, False, True, True, True]`. `result = data[mask]` produces `[30, 40, 50]` via boolean indexing, but *result is never printed*. The f-string prints the original data array and the boolean mask array. NumPy arrays print without commas.

Criterion	Marks
States code executes without error (no exception claimed)	1
Correct output for data part: <code>[10 20 30 40 50]</code> (no commas, NumPy repr)	1
Correct output for mask part: <code>[False False True True True]</code> (capital F/T, no commas)	1

Question 4 — CSS Specificity

/6 marks

Specificity ranking: Inline style (1000) > ID selector (100) > Class selector (10) > Element selector (1)

Sub-part	Answer	Reason	Marks
A. Hello One	Orange	Has inline style → always wins regardless of other selectors	2
B. Hello Two	Orange	Has inline style → wins over .highlight even without an ID	2
C. Hello Three	Blue	No inline style; #main-text (100) beats .highlight (10) beats p (1)	2

Marking note: Full 2 marks awarded for writing the correct colour alone. An explanation is not required. Deduct 1 mark if the colour is correct but accompanied by a contradictory or wrong explanation.

Question 5 — Observer Pattern (JavaScript)

/3 marks

Model Answer:

After `mainSwitch.flip("ON")`, both the Bulb and the Nuclear Reactor are **ON**.

Explanation: When `flip("ON")` is called, it iterates over all devices in `this.devices` and calls each subscriber function with the new state. By the second flip, both the Bulb and the Nuclear Reactor have been added as subscribers, so both receive the "ON" state.

Criterion	Marks
Bulb final state = ON	1
Nuclear Reactor final state = ON	1
Correct explanation: <code>flip()</code> notifies all subscribed observers/devices with the updated state	1

Question 6 — SQL Queries

/15 marks

Grading scale (per sub-question, Anil sir's instructions): 3 (correct or trivially minor errors) | 1.5 (structurally right but major errors, e.g. WHERE instead of HAVING, wrong JOIN syntax) | 0 (fundamental misunderstanding of SQL or too many errors)

A [3 marks] — Employees with salary > 50,000

```
SELECT name
FROM Employees
WHERE salary > 50000;
```

Grader note: Must use WHERE on salary. Accept `e.name`, `Employees.name`. No JOIN needed.

B [3 marks] — Employee names with their department names

```
SELECT e.name, d.dept_name
FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id;
```

Grader note: Must JOIN on `dept_id`. INNER JOIN / JOIN both accepted. Column alias variations accepted.

C [3 marks] — Total salary per department

```
SELECT dept_id, SUM(salary) AS total_salary
FROM Employees
GROUP BY dept_id;
```

Grader note: Must use SUM and GROUP BY `dept_id`. Showing `dept_name` instead requires a JOIN — accept if correct. Using WHERE instead of GROUP BY → 0.

D [3 marks] — Departments where average salary > 50,000

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
HAVING AVG(e.salary) > 50000;
```

Grader note: Must use HAVING (not WHERE) to filter on aggregate. Using WHERE instead of HAVING → 1.5. Missing JOIN but otherwise correct → 1.5.

E [3 marks] — All departments with employee names (including empty departments)

```
SELECT d.dept_name, e.name
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id;
```

Grader note: Must use LEFT JOIN with Departments as the left table to include departments with no employees. Using INNER JOIN → 1.5 (misses the 'no employees' requirement). RIGHT JOIN with Employees on the left → 0 (wrong direction).

Question 7 — Database Type Matching (Social Travel App)**/18 marks**

Marks per feature: Best Fit = 1 pt | Reason = 2 pts | Total per feature = 3 pts

Reason marking: **2/2** — Shows understanding of why this DB fits, even in plain language. Benefit of doubt — if the reason says anything specific about this DB or feature, award 2. **1/2** — So vague it fits any DB ('it is good', 'it is fast'). **0/2** — Factually wrong or describes a different DB's strength.

#	Feature	Best Fit (1 pt)	2/2 — Shows understanding	1/2 — Too vague
1	Auth/Billing	Relational (RDBMS)	All-or-nothing updates; constraints reject invalid data; auto-reversal on failure; prevents orphan records	'secure', 'reliable', 'easy to query'
2	Social Connectivity	Graph	Traverses relationships/follower chains; friend-of-friend lookup; multi-hop links; first-class edges	'faster' or 'good' alone with no elaboration
3	Session Management	Cache	In-memory/RAM → very fast; temporary/short-lived data; auto-expiry; low latency	'fast' or 'good' alone with no elaboration
4	Hotel Listings	Document DB	No fixed schema; each record has different fields; add attributes without restructuring; flexible structure	'easier' or 'good' alone with no elaboration
5	Discovery Search	Inverted Index	Built for keyword/text search; fast keyword lookup; ranks by relevance; natural-language queries	'fast' or 'good' alone with no elaboration
6	Fraud Detection	Graph	Shared IPs/devices as edges; traces account chains (A→IP→B→card→C); detects fraud rings/cycles	'faster' or 'better' alone with no elaboration

Note: Features 2 and 6 both use Graph. If the student gives the same generic reason for both, award 2/2 for the more specific answer and 1/2 for the other. **0/2 examples:** Graph claiming 'ACID integrity'; Cache claiming 'permanent storage'; Document DB claiming 'efficient relationship traversal'.

Question 8 — Code Snippet Outputs (JS & Python)**/12 marks**

3 marks per snippet. For snippets with errors: full marks require both the pre-error output AND the error/exception. Half marks if only one of the two is provided. Deduct 1 mark for each line of output stated that should not appear.

JS-1 [3 marks]

```
const colors = ["Red", "Green", "Blue"];
for (let i = 1; i < colors.length; i++) {
  console.log(colors[i]);
}
console.log(colors[i]); // i is let-scoped – not accessible here
```

Expected output:

```
Green
Blue
ReferenceError: i is not defined
```

Grader note: Loop starts at i=1 so Red is never printed. let is block-scoped to the for loop; accessing i outside the loop throws a ReferenceError. Deduct 1 mark if student incorrectly includes Red in the output.

JS-2 [3 marks]

```
const colors = ["Red", "Green", "Blue"];
for (var i = 1; i < colors.length; i++) {
  console.log(colors[i]);
}
console.log(colors[i]); // var is function-scoped; i=3 after loop; colors[3]=undefined
```

Expected output:

```
Green
Blue
undefined
```

Grader note: Loop starts at i=1 so Red is never printed. var is function-scoped; after the loop i=3. colors[3] is undefined (not an error). Deduct 1 mark for including Red. 0 marks for claiming an error/exception.

Py-1 [3 marks]

```
colors = ["Red", "Green", "Blue"]
for i in range(len(colors)):
  print(colors[i])
```

Expected output:

```
Red
Green
Blue
```

Grader note: range(len(colors)) = range(3) = [0,1,2]. Prints all three elements. No error.

Py-2 [3 marks]

```
colors = ["Red", "Green", "Blue"]
i = 0
while i < len(colors):
  print(colors[i])
  i += 1
print(colors[i]) # i=3 after loop; colors[3] raises IndexError
```

Expected output:

```
Red
Green
Blue
IndexError: list index out of range
```

Grader note: The while loop prints all three elements (i=0,1,2). After the loop i=3. print(colors[3]) raises IndexError. Must state both the three printed lines AND the error for full marks.

Criterion	Marks
Full marks (3/3): correct pre-error output + correct error/exception identified	3

Half marks (1.5/3): only pre-error output stated, or only error identified (not both)	1.5
For snippets with no error (Py-1): all output lines correct	3
Deduction: -1 for each output line stated that should not appear	-1 each

Question 9 — Lambda Closure / Late Binding

/8 marks

```
def create_multipliers():
    return [lambda x: i * x for i in range(4)]
multipliers = create_multipliers()
results = [m(2) for m in multipliers]
print(results)
```

Part (a) — Output [3 marks]

Output: [6, 6, 6, 6]

All four lambdas capture *i* by reference (late binding). By the time any multiplier is called, the loop has completed and *i* = 3. So every call returns $2 \times 3 = 6$.

Criterion	Marks
[6, 6, 6, 6] — full marks	3
Deduction: answer is [8, 8, 8, 8] (<i>i</i> =4, off-by-one error on range)	-2
Deduction: other minor errors in reasoning	-1

Part (b) — Fix the function [5 marks]

Use a default argument to capture the current value of *i* at each iteration:

```
def create_multipliers():
    return [lambda x, i=i: i * x for i in range(4)] # i=i captures current i
multipliers = create_multipliers()
results = [m(2) for m in multipliers]
print(results) # [0, 2, 4, 6]
```

Accepted alternatives: Using `functools.partial`, nested functions/closures that force capture, or any approach that correctly binds each multiplier to its own value of *i*.

Criterion	Marks
Working code using list comprehension with correct multiplier logic — full marks	5
Deduction: no code given, explanation only	-3
Deduction: correct fix but not using list comprehension	-3
Any valid multiplier binding logic accepted (not just default arg)	—

Question 10 — JavaScript Async / Event Loop

/12 marks

```
const threshold = 20;
function buildQuiz() {
  for (var i = 0; i < 2; i++) {
    setTimeout(() => { console.log('A:', i); }, threshold);
  }
  for (let j = 0; j < 2; j++) {
    setTimeout(() => { console.log('B:', j); }, threshold);
  }
  console.log('C: Done');
}
buildQuiz();
```

Part A [3 marks] — Output upon execution

Model output:

```
C: Done
A: 2
A: 2
B: 0
B: 1
```

Criterion	Marks
'C: Done' printed first — synchronous before async	1
'A: 2' printed twice — var hoisting, i=2 by callback time	1
'B: 0' then 'B: 1' — let block-scoping preserves each j	1

Part B [3 marks] — One-line explanation

Model answer: 'C: Done' runs synchronously first; both A callbacks capture the same var-scoped i = 2 (loop already finished); let-scoped j creates a new binding per iteration, so B prints 0 then 1.'

Criterion	Marks
setTimeout is asynchronous — synchronous line 14 (C: Done) executes first	1
var is function-scoped; by callback time the loop is done and i = 2	1
let is block-scoped; each iteration gets its own j binding (0 and 1)	1

Part C [3 marks] — Output if threshold = 0

Model output:

```
C: Done
A: 2
A: 2
B: 0
B: 1
```

Criterion	Marks
Output is identical to Part A — order does not change	1
setTimeout(fn, 0) still defers callbacks to the task queue; sync code runs first	1
var/let scoping unchanged — A still prints 2 twice, B prints 0 then 1	1

Part D [3 marks] — Output if Lines 9 & 11 commented out

Model output:

```
B: 0
B: 1
C: Done
A: 2
A: 2
```

Key insight: Commenting lines 9 & 11 removes only the `setTimeout` wrapper. Line 10 (`console.log('B: ', j)`) remains and now runs synchronously inside the `for`-loop. *Common mistake:* Students may think removing `setTimeout` means no B output — award 0 for Part D if B lines are entirely omitted.

Criterion	Marks
'B: 0' and 'B: 1' first — <code>console.log</code> (line 10) now runs synchronously in the loop	1
'C: Done' after B lines — all synchronous code completes before async callbacks	1
'A: 2' twice last — var-scoped <code>i = 2</code> at callback time (unchanged)	1

Question 11 — Bash Regex Script

/15 marks

Grading: TA discretion based on overall correctness. The script must correctly: (1) read input, (2) define a valid regex with capturing groups for base and exponent, (3) test for a match, (4) extract and print base and exponent separately, (5) handle no-match case.

Reference Solution:

```
#!/bin/bash

# read input from command line into variable 'rawinput'
rawinput="$1"

# regex: base (int or decimal) + e/E indicator + exponent (optional sign + digits)
pattern='^([0-9]+\.[0-9]*)?([eE])([-+]?[0-9]+)$'

# check if there is a match
if [[ $rawinput =~ $pattern ]]; then
    base="${BASH_REMATCH[1]}"
    exponent="${BASH_REMATCH[3]}"
    echo "Base: $base"
    echo "Exponent: $exponent"
else
    echo "No match found: input is not in base-exponent format"
fi
```

Accept: Variations that correctly handle the format (e.g. using `grep -P` with named groups, reading from `stdin` instead of `$1`).

Penalise: Missing the negative exponent case, incorrect group extraction, regex that accepts invalid formats like '2e' or 'e6'.

Criterion	Marks
Reads input correctly (command line or <code>stdin</code>)	3
Valid regex pattern capturing base and exponent in separate groups; handles e/E and negative exponent	6
Correctly tests for match and extracts both groups into variables	3
Prints base and exponent; handles no-match case	3

Question 12 — Web Service Latency & Throughput

/12 marks

Setup: Cache hit rate = 90%. Cache check = 1ms (all requests). DB query = 20ms (miss only). External API = 80ms (miss only). Network transit = 4ms (all requests).

Part A [4 marks] — Expected Average Response Time & Latency

Criterion	Marks
HIT path (90%): Cache check only = 1 ms	1
MISS path (10%): Cache + DB + API = 1 + 20 + 80 = 101 ms	1
System Latency (server-side) = $0.9 \times 1 + 0.1 \times 101 = 0.9 + 10.1 = 11$ ms	1
Expected Average Response Time (client) = Latency + Network transit = 11 + 4 = 15 ms	1

Part B [4 marks] — Server-side Throughput at P = 10

Criterion	Marks
Original throughput = $P / \text{Latency} = 10 / 0.011 \text{ s} \approx 909 \text{ req/s}$	1
New MISS path after cross-continental API (+120 ms): $1 + 20 + (80+120) = 221$ ms	1
New Latency = $0.9 \times 1 + 0.1 \times 221 = 0.9 + 22.1 = 23$ ms	1
New throughput = $10 / 0.023 \approx 435 \text{ req/s}$	1

Part C [4 marks] — Client-side Throughput

Criterion	Marks
Client-side throughput = $P / \text{Response Time} = 10 / 0.015 \approx 667 \text{ req/s}$	1
Server-side $\approx 909 \text{ req/s}$ vs client-side $\approx 667 \text{ req/s}$ — they are not equal	1
In steady state, throughput at both sides converges to the same bottleneck value	1
If network is the bottleneck (4ms transit saturates before server threads), a persistent gap remains — network becomes the limiting resource rather than server capacity	1